

Deploying on Radeon Cloud (ROCm)

Runbook for standing up the vLLM backend once GPU credits are available, so the swap from Ollama to Radeon is copy-paste, not improvised on stream.

0. Platform facts (verified 2026-07-27)

Confirmed by logging in and probing the API directly, so none of this is guesswork:

- **Credits: 5, already in the account.** Granted automatically at signup. There is no application, approval queue or turnaround. The unanswered Jul 16 email to ai_dev_contests@amd.com was never on the critical path.
- **Burn rate: 1 credit per GPU per hour**, stated on the profile page. So the account holds **5 GPU-hours total**, and only one active instance is allowed at a time. Budget the whole submission against that: an fp16 bench, one quantization sweep and the demo recording all have to fit inside 5 hours.
- **Failed launches cost nothing.** Balance held at 5 across four failed attempts.
- **Two launch rate limits per IP**, both returning `template_launch_rate_limited : 3 per 10 minutes` and **5 per hour**. The hourly one is the painful one, its `retry_after` is ~16 minutes. Do not burn launches on smoke tests; each failed boot still counts against both.
- **Boot reliability is not guaranteed.** Qwen/Qwen3-8B booted and ran for an hour without trouble. Qwen/Qwen3-4B-Instruct-2507 loaded its weights (7.67 GiB) and then died twice during or just after `torch.compile`, with the instance reaped and container logs emptied, so no stack trace survives. Suspected platform instability during the announced maintenance rather than a model problem. Prefer the known-good 8B config when credits are tight.
- The platform is showing **"System maintenance is in progress"** and credit redemption is disabled, which is the most likely cause of the hydrate failures below.
- `POST /api/credits/redeem` exists, so a top-up almost certainly happens via a redemption code rather than a support ticket. Ask in Discord for the code if 5 credits turns out to be short.
- The SSH ed25519 public key is already registered on the profile.

Useful endpoints (from `/openapi.json`):

```
POST /api/templates/{id}/launch
GET /api/notebook/status
GET /api/notebook/logs
DELETE /api/notebook/current
POST /api/profile/templates
POST /api/credits/redeem
```

Two operational gotchas:

- A dead instance keeps its slot. Relaunching while one is present silently reuses the old `instance_id` and fails again. Always `DELETE /api/notebook/current`, wait ~10s for `status: not_found`, then relaunch.
- Launch is rate limited: roughly four attempts in a few minutes returns 429. Back off rather than retrying in a loop.

Resolved: the gallery templates cannot serve vLLM. Every `opencode` gallery template failed in about 4 seconds with `error_code: workspace_init_failed`, `reason: WorkspaceInitFailed`, `detail: "Workspace preparation failed in workspace-hydrate"`, reproduced from a clean state (`status: not_found`, no instance), with empty container logs. All 10 gallery templates are `instance_type: opencode` and none serves vLLM.

The fix was to stop using the gallery and create a custom **vLLM Model API** template, which takes a different code path and hydrates normally. Everything measured in this submission ran on an instance created that way.

1. Launch the instance

1. Log in at radeon-global.anruicloud.com (email OTP).
2. Profile -> My Templates -> Add Template.
 - Title: `vulcan-vllm`
 - Container image: an official ROCm + vLLM image (check the template gallery for the current tag; pin it in the demo video credits).
 - Deploy Type: `vLLM Model API`.
 - Serve command:

```
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen3-8B \
  --host 0.0.0.0 --port 8000
```

- Toggle SSH on and upload the `ed25519` public key from Profile if you need a shell for `vulcan bench` and quantization sweeps instead of only the notebook.
3. Launch, then open via SSH or JupyterLab. Instances burn credits from the moment they're running, not from first request, so do all local prep (this doc, the bench script, the demo shot list) before launching.

2. Point vulcan at it

```
export VULCAN_BASE_URL=http://<instance-host>:8000/v1
export VULCAN_MODEL=Qwen/Qwen3-8B
# Do NOT point embeddings at this endpoint. `vllm serve Qwen/Qwen3-8B` starts
# with task=generate and exposes no /v1/embeddings route, so it 404s and
# vulcan/llm.py raises naming this variable. Leave embeddings on a local model:
export VULCAN_EMBED_MODEL=mx-bai-embed-large
export VULCAN_EMBED_BASE_URL=http://localhost:11434/v1
```

No code changes: `vulcan/llm.py` speaks OpenAI-compatible chat against any `base_url`, and keeps a separate client for embeddings precisely so the two can point at different servers, which is what this split needs.

3. Re-index and re-run

```
vulcan index ~/code/some-real-repo
vulcan ask "..."
```

Use a real, moderately sized repo for the demo, not vulcan's own source, so indexing and search look representative on camera.

4. Benchmark and compare

```
vulcan bench --label radeon-vllm-fp16 --repeats 5
```

This writes `bench-results/radeon-vllm-fp16.json`. Compare it against the laptop baseline captured during development:

```
vulcan bench-compare bench-results/ollama-m4air-qwen3-4b.json bench-results/radeon-
vllm-fp16.json
```

Prints a markdown table (TTFT, tokens/sec, delta) straight into the submission doc and demo video. See `vulcan/bench.py:compare`.

4b. Turn off Qwen3 thinking mode

Qwen/Qwen3-8B reasons by default, which is the wrong trade for an agent loop: every ReAct step pays for a reasoning preamble the agent never shows. vLLM exposes the switch per request, no restart and no reserve command change:

```
{"model": "Qwen/Qwen3-8B", "messages": [...], "stream": true,
 "chat_template_kwargs": {"enable_thinking": false}}
```

Measured on the Radeon instance with the bench harness, same prompt, same 5-repeat median:

mode	deltas generated	wall clock	delta rate
thinking (default)	4058	170.9s	23.8/s
<code>enable_thinking: false</code>	1168	49.0s	24.0/s

The generation *rate* is identical. The entire win is that thinking emits ~3.5x more tokens for the same task, so wall-clock latency per agent step falls by the same factor. Worth stating plainly in the writeup: this is a serving-config win, not a claim that the GPU got faster.

5. Quantization sweep (Section 4 of docs/spec.md)

Repeat the serve command with `--quantization awq` or `--quantization int8` (check the image's supported flags) into a second template, bench under a different `--label`, and add a row to the same

comparison table. This is the 40-point ROCm-optimization story for the Track 2 submission: baseline fp16, then a measured quantization tradeoff, not just "it runs".

Credit budget

5 credits confirmed in the account (see Section 0). One credit is one GPU-hour, measured against the running instance: a full benchmark session (boot, model load, four bench runs) fits inside one credit with room to spare. Treat them as scarce anyway: destroy the instance between sessions, don't leave it idle. Sequence per session: launch -> re-index -> ask/chat smoke test -> bench -> destroy.

On the first successful boot, read `credits` from `/api/me` immediately before launching and again five minutes in. That difference is the hourly rate, and it decides whether the quantization sweep in Section 5 is affordable or has to be cut to a single fp16 run.